

Sistema de Controle de Gastos Residenciais

Documentação Técnica e Guia de Arquitetura do Frontend React + TypeScript

Data de geração: 18/07/2026 Versão do projeto: 0.0.0 Branch analisada: main Commit base analisado: ea2f4f0dff258627237d6b0ddab6cc2625791a3c

Sumário

1. Visão geral do sistema
2. Stack tecnológica
3. Arquitetura geral
4. Mapa mental da arquitetura
5. Como a aplicação inicia
6. Providers globais
7. Sistema de rotas
8. Layout principal e layout de autenticação
9. Autenticação
10. Feature de Pessoas
11. E-mail e autenticação
12. Feature de Transações
13. Regra crítica de menores de 18 anos
14. Feature de Resumo Financeiro
15. Total geral
16. Filtros
17. TanStack Query
18. Services e HTTP client
19. Contratos e DTOs
20. React Hook Form + Zod
21. Estados de interface
22. Skeletons
23. ErrorState e 404
24. Análise financeira com IA
25. Animação da IA
26. Material UI e tema
27. SCSS e responsividade
28. Componentes compartilhados
29. Componente Table genérico
30. Gerenciamento de estado
31. Fluxos completos
32. Testes
33. MSW

- 34. Playwright
 - 35. CI/CD
 - 36. Variáveis de ambiente
 - 37. Backend futuro
 - 38. Endpoints esperados
 - 39. Regras de negócio
 - 40. Glossário
 - 41. Guia para estudar o projeto
 - 42. Guia de depuração
 - 43. Como adicionar uma nova feature
 - 44. Boas práticas encontradas
 - 45. Pontos de atenção
 - 46. Checklist de compreensão
 - 47. Apêndice: mapa de arquivos importantes
 - 48. Apêndice: fluxos resumidos
-

1. Visão Geral Do Sistema

O projeto é um frontend React/TypeScript para controle de gastos residenciais. Ele organiza pessoas da casa, registra transações financeiras, separa receitas e despesas, apresenta resumo por pessoa e calcula uma visão geral do saldo. A aplicação também está preparada para autenticação por API REST e para uma análise financeira com IA executada no backend.

O problema resolvido é: permitir que uma residência acompanhe quem participa do orçamento, quais entradas e saídas existem, e qual é o saldo resultante por pessoa e no total geral.

Funcionalidades centrais já implementadas no frontend:

- Pessoas: cadastro, listagem, exclusão e validação de nome, e-mail e idade.
- Transações: cadastro, listagem, exclusão, categorização e tipo `income / expense`.
- Resumo financeiro: consulta por pessoa/período, cards individuais e visão geral.
- Rotas: públicas, privadas, lazy loading, redirecionamento da raiz para pessoas e 404.
- Autenticação: contexto, login, cadastro, logout, guards e bypass de desenvolvimento.
- Estados de interface: loading, erro, vazio e sucesso.
- Testes: unitários, componentes, integração com MSW e E2E com Playwright.

Funcionalidades preparadas para integração:

- Persistência real de pessoas, transações e resumo via API REST.
- Sessão real por cookies, porque o `httpClient` usa `credentials: "include"`.
- Análise financeira com IA via endpoint `/summary/analysis`.
- Regras definitivas de backend, como exclusão em cascata e validação server-side de menores.

Extensões adicionais do projeto:

- Interface de IA no resumo.
- Estados visuais reutilizáveis.
- Responsividade com sidebar desktop e Drawer mobile.
- CI com lint, testes e build.

2. Stack Tecnológica

A stack abaixo foi levantada diretamente de `package.json`.

Tecnologia	Responsabilidade no projeto	Onde é usada
React 19.2.7	Construção da interface em componentes	<code>src/main.tsx</code> , <code>src/features/</code> , <code>src/shared/components/</code>
TypeScript 6.0.2	Tipagem de componentes, hooks, services, DTOs e schemas	<code>tsconfig*.json</code> , arquivos <code>.ts</code> e <code>.tsx</code>
Vite 8.1.1	Dev server, build e ambiente de testes integrado ao Vitest	<code>vite.config.ts</code> , scripts <code>dev</code> , <code>build</code> , <code>preview</code>
Material UI 9.2.0	Componentes visuais e tema	<code>src/app/theme/theme.ts</code> , páginas, layouts e componentes
MUI Icons	Ícones de ações, navegação e feedback	<code>src/shared/components/layout/Template/Template.tsx</code> , páginas de cadastro/listagem, IA
React Router 8.2.0	Rotas, layouts aninhados, <code>Outlet</code> , redirects e navegação	<code>src/app/routes.tsx</code> , <code>src/app/routes/paths.ts</code>
TanStack Query 5.101.2	Cache de dados remotos, queries, mutations e invalidação	hooks em <code>features/people</code> , <code>transactions</code> , <code>summary</code>
React Hook Form 7.82.0	Estado e submissão de formulários	páginas <code>SignIn</code> , <code>SignUp</code> , <code>PersonRegisterPage</code> , <code>TransactionRegisterPage</code>
Zod 4.4.3	Validação declarativa dos formulários	<code>schemas</code> de auth, pessoas e transações
Sass/SCSS	Estilos globais, layouts, responsividade e animações	<code>src/styles</code> , <code>*.scss</code> das páginas/componentes
Vitest 4.1.10	Testes unitários e de componentes	arquivos <code>*.test.ts</code> e <code>*.test.tsx</code>
React Testing Library	Renderização e interação com componentes nos testes	testes de páginas e shared components
MSW 2.15.0	Interceptação de HTTP nos testes	<code>src/test/server.ts</code> , specs com <code>server.use</code>
Playwright 1.61.1	Testes E2E e responsividade em navegador	<code>playwright.config.ts</code> , <code>tests/e2e/auth.spec.ts</code>
ESLint	Análise estática	<code>eslint.config.js</code> , script <code>lint</code>
Prettier	Checagem/formatação de código	scripts <code>format</code> e <code>format:write</code>
GitHub Actions	Pipeline de validação em push/PR	<code>.github/workflows/ci.yml</code>

O projeto não usa Redux. O estado remoto está em TanStack Query, o estado de sessão está no Context de autenticação e os formulários usam React Hook Form.

3. Arquitetura Geral

O projeto é organizado por camadas e por feature. A estrutura real simplificada é:

```
src/  
├── app/  
│   ├── providers/  
│   ├── query/  
│   ├── routes/  
│   ├── routes.tsx  
│   └── theme/  
├── assets/  
│   └── images/  
├── features/  
│   ├── authentication/  
│   │   ├── components/  
│   │   ├── contexts/  
│   │   ├── hooks/  
│   │   ├── pages/  
│   │   ├── providers/  
│   │   ├── schemas/  
│   │   ├── services/  
│   │   └── types/  
│   ├── not-found/  
│   ├── people/  
│   │   ├── hooks/  
│   │   ├── pages/  
│   │   ├── schemas/  
│   │   ├── services/  
│   │   └── types/  
│   ├── summary/  
│   │   ├── components/  
│   │   ├── hooks/  
│   │   ├── pages/  
│   │   ├── services/  
│   │   ├── types/  
│   │   └── utils/  
│   └── transactions/  
│       ├── hooks/  
│       ├── pages/  
│       ├── schemas/  
│       ├── services/  
│       ├── types/  
│       └── utils/  
├── shared/  
│   ├── api/  
│   ├── components/  
│   └── config/  
├── styles/  
└── test/
```

`src/app` concentra infraestrutura de aplicação: providers, roteamento, query client e tema. `src/shared` guarda recursos reutilizáveis e independentes de uma feature específica: HTTP client, erros, componentes de layout, tabela, skeletons e estados. `src/features` agrupa domínio e telas por funcionalidade. Isso reduz

acoplamento: Pessoas não precisa conhecer detalhes internos de Resumo, exceto quando hooks são reutilizados de forma explícita, como `SummaryPage` usando `usePeople` para preencher filtros.

4. Mapa Mental Da Arquitetura

Fluxo conceitual de leitura:

```
flowchart TD
  U[Usuário] --> P[Página React]
  P --> H[Hook da feature]
  H --> Q[TanStack Query]
  Q --> S[Service]
  S --> C[httpClient]
  C --> A[API REST]
  A --> B[Backend futuro]
```

Retorno dos dados:

```
flowchart TD
  A[API REST] --> C[httpClient]
  C --> S[Service]
  S --> D[Mapper DTO]
  D --> Q[Query Cache]
  Q --> P[Componente]
  P --> I[Interface]
```

Representação textual para leitura direta no PDF:

```
Usuário
↓
Página React
↓
Hook da feature
↓
TanStack Query
↓
Service
↓
httpClient
↓
API REST
↓
Backend futuro
```

Esse padrão aparece, por exemplo, em Pessoas:

```
src/features/people/pages/PeopleConsultPage/PeopleConsultPage.tsx →
src/features/people/hooks/usePeople.ts → src/features/people/services/peopleService.ts →
src/shared/api/httpClient.ts → /people.
```

5. Como A Aplicação Inicia

Entrada real: `src/main.tsx`.

Fluxo:

1. O navegador carrega `index.html`.
2. Vite executa `src/main.tsx`.
3. `createRoot(document.getElementById("root")!)` cria a raiz React.
4. A aplicação entra em `StrictMode`.
5. `AppProviders` adiciona Query, tema Material UI, reset CSS e autenticação.
6. `RouterProvider` recebe o `router` de `src/app/routes.tsx`.
7. O React Router resolve a URL atual.
8. O guard público ou privado decide se libera ou redireciona.
9. O layout (`Template` ou `AuthTemplate`) renderiza.
10. A página filha aparece dentro do `<Outlet />`.

Snippet:

```
createRoot(document.getElementById("root")!).render(  
  <StrictMode>  
    <AppProviders>  
      <RouterProvider router={router} />  
    </AppProviders>  
  </StrictMode>,  
);
```

Por blocos:

- `StrictMode` ajuda o React a detectar efeitos colaterais no desenvolvimento.
- `AppProviders` concentra dependências globais.
- `RouterProvider` entrega a árvore de rotas declarada em `src/app/routes.tsx`.

Fluxograma:

```
index.html
↓
src/main.tsx
↓
AppProviders
↓
RouterProvider
↓
Guard da rota
↓
Layout
↓
Página
```

6. Providers Globais

Arquivo: `src/app/providers/AppProviders.tsx`.

Árvore real:



Responsabilidades:

Provider	Arquivo	Responsabilidade
QueryClientProvider	<code>src/app/providers/AppProviders.tsx</code>	Disponibiliza TanStack Query para hooks de dados remotos
ThemeProvider	<code>src/app/providers/AppProviders.tsx</code>	Aplica o tema MUI declarado em <code>src/app/theme/theme.ts</code>
CssBaseline	<code>src/app/providers/AppProviders.tsx</code>	Normaliza estilos base do Material UI
AuthProvider	<code>src/features/authentication/providers/AuthProvider.tsx</code>	Mantém usuário, sessão e ações de autenticação
RouterProvider	<code>src/main.tsx</code>	Renderiza rotas declaradas em <code>src/app/routes.tsx</code>

A ordem permite que páginas renderizadas por rotas usem tanto o contexto de autenticação quanto TanStack Query e componentes MUI tematizados.

7. Sistema De Rotas

Arquivos principais:

- `src/app/routes.tsx`
- `src/app/routes/paths.ts`
- `src/app/routes/pageTitles.ts`
- `src/features/authentication/components/PrivateRoute.tsx`
- `src/features/authentication/components/PublicRoute.tsx`

Tabela de rotas reais:

Rota	Página/Layout	Tipo	Objetivo
<code>/entrar</code>	<code>SignIn</code> dentro de <code>AuthTemplate</code>	Pública	Login
<code>/cadastrar</code>	<code>SignUp</code> dentro de <code>AuthTemplate</code>	Pública	Cadastro de conta
<code>/</code>	<code>Navigate</code> para <code>/pessoas</code> dentro de <code>Template</code>	Privada	Entrada padrão da área autenticada
<code>/pessoas</code>	<code>PeopleConsultPage</code>	Privada	Listar/excluir pessoas
<code>/pessoas/registrar</code>	<code>PersonRegisterPage</code>	Privada	Cadastrar pessoa
<code>/transacoes</code>	<code>TransactionsConsultPage</code>	Privada	Listar/excluir transações
<code>/transacoes/registrar</code>	<code>TransactionRegisterPage</code>	Privada	Cadastrar transação
<code>/resumo</code>	<code>SummaryPage</code>	Privada	Consultar resumo financeiro e IA
<code>*</code>	<code>NotFoundPage</code>	Pública/fallback	Exibir 404

`src/app/routes.tsx` usa `lazy` e `Suspense`; enquanto uma página carregada dinamicamente chega, o fallback é `PageSkeleton`.

Fluxo real de navegação:

```
URL
↓
createBrowserRouter
↓
PublicRoute ou PrivateRoute
↓
AuthTemplate ou Template
↓
Outlet
↓
Página lazy carregada
```

`PublicRoute` redireciona usuários autenticados para `/pessoas`. `PrivateRoute` redireciona usuários não autenticados para `/entrar`, preservando `location` no state. Se `VITE_BYPASS_AUTH=true`, ambos retornam `<Outlet />` sem bloquear a navegação.

8. Layout Principal E Layout De Autenticação

Template Principal

Arquivo: `src/shared/components/layout/Template/Template.tsx`.

O `Template` é o layout da área privada. Ele contém:

- sidebar desktop com avatar, nome, e-mail e navegação;
- navegação por MUI `Tabs`;
- ícones para Pessoas, Transações e Resumo;
- logout;
- header mobile com botão de menu;
- MUI `Drawer` em telas menores que `md`;
- área principal com `<Outlet />`;
- rodapé/assinatura visual com logo.

O `<Outlet />` é o ponto em que a rota filha é renderizada. Quando a URL é `/pessoas`, o `Template` permanece e o conteúdo interno vira `PeopleConsultPage`. Quando é `/resumo`, o mesmo layout permanece e o conteúdo interno vira `SummaryPage`.

AuthTemplate

Arquivo: `src/shared/components/layout/AuthTemplate/AuthTemplate.tsx`.

Estrutura:

```
AuthTemplate
├── left-container
│   ├── logo
│   ├── título
│   ├── descrição
│   └── imagem preview
└── right-container
    ├── Outlet
    │   ├── SignIn
    │   └── SignUp
```

Ele é usado apenas pelas rotas públicas `/entrar` e `/cadastrar`.

9. Autenticação

Arquivos:

- `src/features/authentication/providers/AuthProvider.tsx`
- `src/features/authentication/contextts/AuthContext.ts`
- `src/features/authentication/hooks/useAuth.ts`
- `src/features/authentication/services/authenticationService.ts`
- `src/features/authentication/components/PrivateRoute.tsx`
- `src/features/authentication/components/PublicRoute.tsx`
- `src/features/authentication/pages/SignIn/SignIn.tsx`
- `src/features/authentication/pages/SignUp/SignUp.tsx`
- `src/features/authentication/schemas/authSchemas.ts`

Implementado No Frontend

`AuthProvider` mantém:

- `user;`
- `isLoading;`
- `isAuthUnavailable;`
- `isAuthenticated;`
- `signIn;`
- `signUp;`
- `signOut.`

Ao montar, se `VITE_BYPASS_AUTH` não estiver ativo, ele chama `authenticationService.getCurrentUser()`. Se a API responder com usuário, o estado passa a autenticado. Se houver erro de rede ou timeout, `isAuthUnavailable` é marcado.

`SignIn` usa `signInSchema`, chama `signIn(data)` e navega para `/pessoas` em caso de sucesso. `SignUp` usa `signUpSchema`, chama `signUp(data)` e também navega para `/pessoas`.

Dependente De Backend

A autenticação real depende de endpoints compatíveis:

- `POST /auth/signin`
- `POST /auth/signup`
- `POST /auth/signout`
- `GET /auth/me`

O frontend não implementa geração de token, senha, hash, cookie, refresh token ou autorização server-side. Ele apenas chama a API e mantém o usuário retornado em memória.

Bypass De Desenvolvimento

Arquivo: `src/shared/config/env.ts`.

```
VITE_BYPASS_AUTH=true
→ PrivateRoute libera acesso
→ PublicRoute libera acesso
→ AuthProvider não tenta carregar usuário real

VITE_BYPASS_AUTH=false
→ fluxo normal de autenticação
→ PrivateRoute exige usuário autenticado
→ PublicRoute redireciona usuário já autenticado
```

Esse bypass é recurso de desenvolvimento/teste. Ele não representa autenticação real e não substitui backend.

10. Feature De Pessoas

Arquivos principais:

- `src/features/people/pages/PeopleConsultPage/PeopleConsultPage.tsx`
- `src/features/people/pages/PersonRegisterPage/PersonRegisterPage.tsx`
- `src/features/people/hooks/usePeople.ts`
- `src/features/people/services/peopleService.ts`
- `src/features/people/schemas/personSchema.ts`
- `src/features/people/types/person.ts`
- `src/features/people/types/personDtos.ts`

Modelo

Person possui:

- `id: number`
- `name: string`
- `email: string`
- `age: number`

`CreatePersonInput` envia apenas `name`, `email` e `age`. O `id` vem da API.

Listagem

Fluxo real:

```
PeopleConsultPage
  ↓
usePeople
  ↓
useQuery(["people"])
  ↓
peopleService.getPeople
  ↓
httpClient.get("/people")
  ↓
API
```

Na interface:

- `loading: TableSkeleton;`
- `erro: ErrorState` com `retry` via `refetch`;
- `vazio: EmptyState;`

- sucesso: `Table<Person>`.

Cadastro

Fluxo:

```
Form
  ↓
React Hook Form
  ↓
Zod personSchema
  ↓
useCreatePerson mutation
  ↓
peopleService.createPerson
  ↓
POST /people
  ↓
invalidateQueries(["people"])
  ↓
navigate("/pessoas")
```

Snippet:

```
export const personSchema = z.object({
  name: z.string().min(1, "Informe o nome."),
  email: z.email("Informe um e-mail válido."),
  age: z.number().int().min(1).max(120),
});
```

Esse schema impede cadastro sem nome, com e-mail inválido ou idade fora da faixa esperada.

Exclusão

Fluxo:

```
Usuário clica Excluir
  ↓
window.confirm
  ↓
useDeletePerson mutation
  ↓
DELETE /people/:id
  ↓
invalidateQueries(["people"])
  ↓
invalidateQueries(["transactions"])
  ↓
invalidateQueries(["summary"])
```

A mensagem de confirmação informa que as transações da pessoa também serão removidas. No frontend, a responsabilidade é confirmar e chamar a exclusão da pessoa. No backend futuro, a exclusão em cascata deve ser garantida definitivamente.

11. E-Mail E Autenticação

O tipo `Person` contém `email`, com comentário explícito em `src/features/people/types/person.ts`: o e-mail fica no cadastro de pessoa porque também será usado como credencial de autenticação.

Isso cria uma ponte entre domínio e autenticação:

- dados de domínio da Pessoa: nome, idade e e-mail de identificação;
- dados de autenticação: e-mail e senha.

A senha não aparece em `Person`. Ela está restrita aos tipos e schemas de autenticação:

- `SignInCredentials`;
- `SignUpData`;
- `signInSchema`;
- `signUpSchema`;
- `SignUpRequestDTO`, que remove `confirmPassword`.

Essa separação é importante: a pessoa pode ter e-mail como atributo do domínio, mas a senha deve permanecer isolada no fluxo de autenticação e ser tratada pelo backend com segurança.

12. Feature De Transações

Arquivos:

- `src/features/transactions/pages/TransactionsConsultPage/TransactionsConsultPage.tsx`
- `src/features/transactions/pages/TransactionRegisterPage/TransactionRegisterPage.tsx`
- `src/features/transactions/hooks/useTransactions.ts`
- `src/features/transactions/services/transactionsService.ts`
- `src/features/transactions/schemas/transactionSchema.ts`
- `src/features/transactions/types/transaction.ts`
- `src/features/transactions/types/transactionDtos.ts`
- `src/features/transactions/utils/transactionLabels.ts`

`Transaction` possui:

- `id;`
- `personId;`
- `personName;`
- `description;`
- `category;`
- `type;`
- `value;`
- `date;`
- `observation.`

Tipos internos:

```
export type TransactionType = "income" | "expense";
```

Labels em português:

```
export const transactionTypeLabels = {  
  income: "Receita",  
  expense: "Despesa",  
};
```

Manter valores internos em inglês é comum em contratos técnicos, enquanto a UI apresenta termos em português. O importante é ter mapeamento centralizado, como em `src/features/transactions/utils/transactionLabels.ts`.

Listagem

`TransactionsConsultPage` chama `useTransactions()` sem filtros. A página renderiza:

- `TableSkeleton` enquanto carrega;
- `ErrorState` se falhar;
- `EmptyState` se a API devolver lista vazia;
- `Table<Transaction>` quando há dados.

Cadastro

`TransactionRegisterPage` carrega pessoas com `usePeople`, porque uma transação precisa pertencer a uma pessoa válida. Depois valida o formulário com `transactionSchema` e envia por `useCreateTransaction`.

13. Regra Crítica De Menores De 18 Anos

Regra:

```
Pessoa < 18
  → somente Despesa

Pessoa >= 18
  → Receita ou Despesa
```

Implementação no frontend:

`src/features/transactions/pages/TransactionRegisterPage/TransactionRegisterPage.tsx`.

O formulário observa `personId` e `type` com `useWatch`. A pessoa selecionada é buscada na lista carregada por `usePeople`.

Quando a pessoa selecionada tem menos de 18 anos e o tipo atual é `income`, um `useEffect` troca o tipo para `expense`:

```
if (isSelectedPersonUnderAge && selectedType === "income") {
  setValue("type", "expense", {
    shouldDirty: true,
    shouldTouch: true,
    shouldValidate: true,
  });
}
```

Além disso, a opção `Receita` fica desabilitada no `Select`:

```
<MenuItem value="income" disabled={isSelectedPersonUnderAge}>
  Receita
</MenuItem>
```

Antes de enviar, o `onSubmit` valida novamente:

```
if (person?.age !== undefined && person.age < 18 && data.type === "income") {
  setSubmitError("Pessoas menores de 18 anos só podem ter despesas cadastradas.");
  return;
}
```

Fluxo visual:


```
Seleciona pessoa
  ↓
idade < 18?
  ✓      \
sim      não
  ↓      ↓
só despesa receita/despesa
```

Frontend = UX e prevenção. Backend futuro = garantia definitiva, porque regras críticas não podem depender apenas do navegador.

Teste existente:

`src/features/transactions/pages/TransactionRegisterPage/TransactionRegisterPage.test.tsx` valida que uma receita escolhida antes é corrigida para despesa quando a pessoa menor é selecionada.

14. Feature De Resumo Financeiro

Arquivos:

- `src/features/summary/pages/SummaryPage.tsx`
- `src/features/summary/components/SummaryFilters.tsx`
- `src/features/summary/components/PersonSummaryCard.tsx`
- `src/features/summary/components/OverviewPanel.tsx`
- `src/features/summary/components/SummarySkeleton.tsx`
- `src/features/summary/hooks/useSummary.ts`
- `src/features/summary/services/summaryService.ts`
- `src/features/summary/types/summary.ts`
- `src/features/summary/types/summaryDtos.ts`
- `src/features/summary/utils/currency.ts`

Fluxo:

```
SummaryPage
↓
SummaryFilters
↓
useSummary(filters)
↓
summaryService.getSummary(filters)
↓
GET /summary?personId=&startDate=&endDate=
↓
PersonSummaryCard + OverviewPanel
```

`PersonSummary` contém:

- `personId`;
- `personName`;
- `income`;
- `expenses` .

Saldo:

```
saldo = receita - despesa
```

`PersonSummaryCard` calcula o saldo individual. `OverviewPanel` calcula o total geral.

15. Total Geral

Arquivo: `src/features/summary/components/OverviewPanel.tsx`.

A API retorna dados individuais por pessoa. O frontend agrega:

- `totalIncome`;
- `totalExpenses`;
- `totalBalance`.

Snippet:

```
const totalIncome = summary.reduce(
  (total, person) => total + person.income,
  0,
);
const totalExpenses = summary.reduce(
  (total, person) => total + person.expenses,
  0,
);
const totalBalance = totalIncome - totalExpenses;
```

Exemplo:

Pessoa	Receita	Despesa	Saldo
Pessoa A	1000	400	600
Pessoa B	500	200	300
Total	1500	600	900

Não existe necessariamente endpoint separado para total geral porque o frontend já tem os totais individuais necessários para somar.

16. Filtros

Filtros reais do resumo:

- pessoa;
- data inicial;
- data final.

Arquivos:

- `src/features/summary/components/SummaryFilters.tsx`
- `src/features/summary/pages/SummaryPage.tsx`
- `src/features/summary/hooks/useSummary.ts`
- `src/features/summary/services/summaryService.ts`

Fluxo:

```
UI
  ↓
estado local filters
  ↓
queryKey ["summary", filters]
  ↓
summaryService
  ↓
params HTTP
```

`personId: "all"` não é enviado para a API. O service transforma `"all"` em `undefined`.

Filtros entram na query key porque resultados diferentes precisam de caches diferentes. Resumo de todas as pessoas não é a mesma consulta que resumo da pessoa 1 em julho.

17. TanStack Query

Arquivo base: `src/app/query/queryClient.ts`.

Configuração:

```
export const queryClient = new QueryClient({
  defaultOptions: {
    queries: {
      retry: 1,
      refetchOnWindowFocus: false,
      staleTime: 1000 * 60,
    },
  },
});
```

No projeto:

- Query = leitura de dados: pessoas, transações, resumo.
- Mutation = escrita/ação: criar, atualizar, excluir, analisar com IA.
- Query key = identidade da consulta no cache.
- Cache = memória gerenciada pelo TanStack Query.
- `invalidateQueries` = marca dados como desatualizados para recarregar.

Exemplos reais:

Hook	Tipo	Query key	Service
<code>usePeople</code>	Query	<code>["people"]</code>	<code>peopleService.getPeople</code>
<code>useCreatePerson</code>	Mutation	invalida <code>["people"]</code>	<code>peopleService.createPerson</code>
<code>useTransactions</code>	Query	<code>["transactions", filters]</code>	<code>transactionsService.getTransactions</code>
<code>useCreateTransaction</code>	Mutation	invalida <code>["transactions"]</code> e <code>["summary"]</code>	<code>transactionsService.createTransaction</code>
<code>useSummary</code>	Query	<code>["summary", filters]</code>	<code>summaryService.getSummary</code>
<code>useFinancialAnalysis</code>	Mutation	sem cache de lista	<code>financialAnalysisService.analyze</code>

`useSummary` usa `placeholderData: keepPreviousData`, então ao alterar filtros ele pode manter os dados anteriores enquanto busca os novos. A UI mostra `Atualizando dados...` quando `isFetching` e já existe resumo na tela.

18. Services E HTTP Client

Componentes não chamam `fetch` diretamente. O caminho adotado é:



`src/shared/api/httpClient.ts` centraliza:

- base URL vinda de `VITE_API_URL`;
- montagem de query string;
- métodos `get`, `post`, `put`, `delete`;
- `Content-Type: application/json`;
- `credentials: "include"`;
- timeout padrão de 10 segundos;
- tratamento de status HTTP;
- conversão para `ApiError`.

Services reais:

Service	Responsabilidade
<code>authenticationService</code>	Login, cadastro, logout e usuário atual
<code>peopleService</code>	Listar, buscar por id, criar, atualizar e excluir pessoas
<code>transactionsService</code>	Listar com filtros, buscar por id, criar, atualizar e excluir transações
<code>summaryService</code>	Buscar resumo financeiro com filtros
<code>financialAnalysisService</code>	Solicitar análise financeira por IA

`financialAnalysisService` usa timeout de 45 segundos, porque análise por IA pode demorar mais que uma consulta comum.

19. Contratos E DTOs

O projeto separa tipos de domínio de DTOs.

Tipos de domínio ficam em arquivos como:

- `src/features/people/types/person.ts`
- `src/features/transactions/types/transaction.ts`
- `src/features/summary/types/summary.ts`
- `src/features/summary/types/financialAnalysis.ts`
- `src/features/authentication/types/auth.ts`

DTOs ficam em:

- `src/features/people/types/personDtos.ts`
- `src/features/transactions/types/transactionDtos.ts`
- `src/features/summary/types/summaryDtos.ts`
- `src/features/summary/types/financialAnalysisDtos.ts`
- `src/features/authentication/types/authDtos.ts`

Exemplo Pessoas:

```
CreatePersonRequestDTO
```

```
→ name  
→ email  
→ age
```

```
PersonResponseDTO
```

```
→ id  
→ name  
→ email  
→ age
```

O formulário de criação não envia `id` automático porque o identificador deve ser responsabilidade do backend/banco de dados. O frontend envia intenção de criação; a API responde com a entidade criada.

Mappers como `mapPersonResponseToPerson` e `mapTransactionResponseToTransaction` protegem o restante da UI de mudanças diretas no formato da resposta.

20. React Hook Form + Zod

Fluxo padrão:

```
Usuário digita
↓
React Hook Form controla campos
↓
Zod valida via zodResolver
↓
handleSubmit chama onSubmit
↓
mutation ou auth action
↓
service
↓
API
```

Onde aparece:

Formulário	Arquivo	Schema
Login	<code>src/features/authentication/pages/SignIn/SignIn.tsx</code>	<code>signInSchema</code>
Cadastro de conta	<code>src/features/authentication/pages/SignUp/SignUp.tsx</code>	<code>signUpSchema</code>
Cadastro de pessoa	<code>src/features/people/pages/PersonRegisterPage/PersonRegisterPage.tsx</code>	<code>personSchema</code>
Cadastro de transação	<code>src/features/transactions/pages/TransactionRegisterPage/TransactionRegisterPage.tsx</code>	<code>transactionSchema</code>

Conceitos aplicados:

- `register`: conecta campos simples ao formulário.
- `Controller`: integra componentes controlados do MUI, como `Select`.
- `handleSubmit`: roda validação e chama `onSubmit` com dados válidos.
- `formState.errors`: alimenta `helperText` e `error`.
- `isSubmitting`: controla botão de envio/loading.

No cadastro de transação, campos numéricos usam `valueAsNumber` para entregar número ao Zod.

21. Estados De Interface

Padrão adotado:

```
Loading
  → Skeleton

Sucesso sem dados
  → EmptyState

Erro
  → ErrorState

Sucesso com dados
  → Conteúdo
```

Isso aparece em Pessoas, Transações e Resumo.

Por que separar:

- loading significa que ainda não há resposta;
- vazio significa que a resposta foi bem sucedida, mas não trouxe itens;
- erro significa falha técnica ou HTTP;
- sucesso significa que a tela pode renderizar os dados reais.

Componentes compartilhados:

- `src/shared/components/DataState/EmptyState.tsx`
- `src/shared/components/DataState/ErrorState.tsx`
- `src/shared/components/ErrorMessage/ErrorMessage.tsx`
- `src/shared/components/FeedbackState/FeedbackState.tsx`
- `src/shared/components/skeletons/TableSkeleton.tsx`
- `src/shared/components/skeletons/PageSkeleton.tsx`
- `src/shared/components/skeletons/FormSkeleton.tsx`

22. Skeletons

Skeletons são usados para evitar telas vazias durante carregamentos.

Skeleton	Arquivo	Uso
PageSkeleton	src/shared/components/skeletons/PageSkeleton.tsx	fallback de lazy routes e guards carregando
TableSkeleton	src/shared/components/skeletons/TableSkeleton.tsx	listagens de pessoas/transações
FormSkeleton	src/shared/components/skeletons/FormSkeleton.tsx	componente reutilizável para formulários, embora não seja central nos fluxos lidos
SummarySkeleton	src/features/summary/components/SummarySkeleton.tsx	loading inicial do resumo

No resumo existe diferença entre loading inicial e refetch. Se `isLoading` e ainda não há dados, aparece `SummarySkeleton`. Se já há dados e uma nova busca está acontecendo, a página mostra `Atualizando dados...`.

23. ErrorState E 404

`src/shared/api/apiError.ts` transforma erros técnicos em mensagens amigáveis. Por exemplo, erro de rede vira título contextual e descrição como "Tente novamente em alguns instantes."

`ErrorState` é usado para falhas de carregamento ou ações:

- lista de pessoas;
- lista de transações;
- opções de pessoas no cadastro de transação;
- resumo;
- análise financeira.

`NotFoundPage`, em `src/features/not-found/pages/NotFoundPage/NotFoundPage.tsx`, trata rota inexistente. Ela usa `FeedbackState`, imagem própria e botão para ir ao resumo.

Diferença:

- erro da aplicação: rota existe, mas dados ou ação falharam;
- 404: a URL não corresponde a nenhuma rota declarada.

24. Análise Financeira Com IA

Arquivos:

- `src/features/summary/components/AiAnalysisCard.tsx`
- `src/features/summary/components/FinancialAnalysisDialog.tsx`
- `src/features/summary/components/FinancialAnalysisLoading.tsx`
- `src/features/summary/components/FinancialAnalysisResult.tsx`
- `src/features/summary/hooks/useFinancialAnalysis.ts`
- `src/features/summary/services/financialAnalysisService.ts`
- `src/features/summary/types/financialAnalysis.ts`
- `src/features/summary/types/financialAnalysisDtos.ts`

Finalidade esperada:

- analisar transações;
- gerar resumo;
- identificar padrões;
- destacar pontos positivos;
- apontar pontos de atenção;
- sugerir recomendações educativas.

Fluxo:

```
Usuário clica "Analisar transações"
↓
SummaryPage abre FinancialAnalysisDialog
↓
useFinancialAnalysis mutation
↓
financialAnalysisService.analyze
↓
POST /summary/analysis
↓
backend processa dados e IA
↓
FinancialAnalysisResult renderiza resposta
```

Antes de chamar a API, `SummaryPage` verifica se existe algum dado financeiro (`income > 0` ou `expenses > 0`). Se não houver, abre o modal com estado de dados insuficientes e não chama `/summary/analysis`.

Frontend implementado:

- card de chamada para ação;
- modal;

- loading animado;
- request para API;
- renderização do resultado;
- erro amigável e retry;
- contexto visual de pessoa/período.

Backend dependente:

- consulta definitiva das transações;
 - execução da IA;
 - geração de `summary`, `positives`, `warnings` e `recommendations`;
 - segurança e limites de uso.
-

25. Animação Da IA

Arquivos:

- `src/features/summary/components/FinancialAnalysisLoading.tsx`
- `src/features/summary/components/FinancialAnalysisDialog.scss`

A animação é construída com React + SCSS. Não é vídeo em runtime. O componente renderiza elementos HTML representando núcleo, linhas de análise e gráfico. O SCSS aplica `@keyframes`:

- `analysis-sweep;`
- `analysis-pulse;`
- `analysis-row;`
- `analysis-line;`
- `analysis-bar.`

Também existe suporte a acessibilidade:

- `role="status";`
 - `aria-label="Analisando suas finanças";`
 - `@media (prefers-reduced-motion: reduce)` reduz animações.
-

26. Material UI E Tema

Tema: `src/app/theme/theme.ts`.

O tema define:

- paleta primária `#310026`;
- success `#2e7d32`;
- error `#d32f2f`;
- background default/paper;
- fonte Poppins;
- botões sem sombra;
- `TextField` e `FormControl` com `size="small"`;
- border radius padrão 8.

MUI aparece em:

- layouts (`Drawer`, `Tabs`, `Avatar`);
- formulários (`TextField`, `Select`, `FormControl`);
- tabela (`Table`, `TableRow`, `TableCell`);
- feedback (`Alert`, `Snackbar`, `Dialog`, `Button`);
- ícones.

Relação MUI + SCSS:

MUI

→ tokens, componentes e interações prontas

SCSS

→ layout, responsividade, grids, animações e ajustes específicos

27. SCSS E Responsividade

Arquivos globais:

- `src/styles/global.scss`
- `src/styles/_breakpoints.scss`

Breakpoints:

```
$breakpoint-sm: 600px;  
$breakpoint-md: 900px;  
$breakpoint-lg: 1200px;
```

Mixin:

```
@mixin down($breakpoint) {  
  @media (max-width: $breakpoint) {  
    @content;  
  }  
}
```

Responsividade observada:

- `Template` usa `useMediaQuery(theme.breakpoints.down("md"))`.
- Desktop: sidebar fixa em `<aside>`.
- Mobile/tablet: header compacto e MUI `Drawer`.
- Tabelas têm `overflowX: "auto"` e `minWidth: 640`.
- `Summary` usa layout com conteúdo e aside, adaptado por SCSS.
- `AuthTemplate` separa área visual e área de formulário.

28. Componentes Compartilhados

Componente	Responsabilidade	Onde é usado
Template	Layout privado com navegação	rotas privadas em <code>src/app/routes.tsx</code>
AuthTemplate	Layout de login/cadastro	rotas públicas em <code>src/app/routes.tsx</code>
Table<T>	Tabela genérica com colunas e ações	Pessoas e Transações
PageHeader	Breadcrumb e título da página	páginas principais
PageTitle	Atualiza <code>document.title</code>	wrapper em <code>src/app/routes.tsx</code>
EmptyState	Estado sem dados	Pessoas, Transações, Resumo
ErrorState	Estado de erro com imagem/ação	listas, resumo e IA
FeedbackState	Base visual para erro/404	<code>ErrorState</code> , <code>NotFoundPage</code>
PageSkeleton	Loading de página	lazy routes e guards
TableSkeleton	Loading de tabela	Pessoas e Transações
FormSkeleton	Loading de formulário reutilizável	disponível em shared

29. Componente Table Genérico

Arquivo: `src/shared/components/Table/Table.tsx`.

`Table` é genérico: `const Table = <T,>(...)`. Isso permite usar a mesma tabela com `Person`, `Transaction` ou outro tipo futuro.

Cada coluna pode ser:

- chave direta de `T`, como `name` em `Pessoas`;
- coluna renderizada por função, como `person` em `Transações`, que exibe `transaction.personName`.

Ações são passadas por lista:

```
interface TableAction<T> {  
  label: string;  
  icon: ReactNode;  
  color?: "inherit" | "primary" | "secondary" | "error" | "info" | "success" | "warning";  
  onClick: (row: T) => void;  
}
```

Isso evita duplicar estrutura de tabela nas features.

30. Gerenciamento De Estado

Tipos de estado no projeto:

Tipo	Ferramenta	Exemplos
Estado local	<code>useState</code>	drawer mobile, filtros do resumo, feedbacks, modal de IA
Formulários	React Hook Form	login, cadastro, pessoa, transação
Estado remoto	TanStack Query	pessoas, transações, resumo
Sessão	Context API	<code>AuthProvider</code> e <code>useAuth</code>

Redux não é necessário neste momento porque não há estado global complexo compartilhado entre muitas áreas. O que é remoto fica em cache de query; o que é sessão fica em contexto; o que é temporário fica local.

31. Fluxos Completos

Cadastrar Pessoa

1. Usuário acessa `/pessoas/regar`.
2. `PrivateRoute` libera a rota se autenticado ou se bypass estiver ativo.
3. `Template` renderiza `<Outlet />`.
4. `PersonRegisterPage` aparece.
5. React Hook Form inicia com `name`, `email` e `age`.
6. `personSchema` valida os dados.
7. `onSubmit` chama `createPerson.mutateAsync(data)`.
8. `peopleService.createPerson` faz `POST /people`.
9. `useCreatePerson` invalida `["people"]`.
10. A página navega para `/pessoas` com mensagem de sucesso.
11. `PeopleConsultPage` lê o feedback do state da rota e mostra `Snackbar`.

Cadastrar Transação

1. Usuário acessa `/transacoes/regar`.
2. `TransactionRegisterPage` chama `usePeople`.
3. Se não houver pessoas, exibe `EmptyState` e desabilita salvar.
4. Usuário seleciona uma pessoa.
5. O formulário identifica idade da pessoa.
6. Se menor de 18 anos, `Receita` fica desabilitada e o tipo é corrigido para `expense`.
7. `transactionSchema` valida campos obrigatórios e valor positivo.
8. `onSubmit` valida novamente a regra de menor.
9. `useCreateTransaction` chama `transactionsService.createTransaction`.
10. O service faz `POST /transactions`.
11. A mutation invalida `["transactions"]` e `["summary"]`.
12. A página navega para `/transacoes`.

Consultar Resumo

1. Usuário acessa `/resumo`.
2. `SummaryPage` inicia filtros com `personId: "all"`, datas vazias.
3. `usePeople` carrega pessoas para o filtro.
4. `useSummary(filters)` busca `/summary`.

5. `summaryService` transforma `"all"` em ausência de `personId`.
6. Cards individuais são renderizados por `PersonSummaryCard`.
7. `OverviewPanel` soma receitas/despesas e calcula saldo total.
8. Alterar filtros muda a query key e dispara nova busca.

Login

1. Usuário acessa `/entrar`.
2. `PublicRoute` libera a rota se não autenticado ou se bypass ativo.
3. `AuthTemplate` renderiza `SignIn`.
4. Formulário valida `email` e `password` com `signInSchema`.
5. `signIn(data)` chama `authenticationService.signIn`.
6. O service faz `POST /auth/signin`.
7. `AuthProvider` salva `user`.
8. A página navega para `/pessoas`.

Análise Com IA

1. Usuário acessa `/resumo`.
 2. Resumo é carregado.
 3. Usuário clica `Analisar transações`.
 4. Se não houver dados financeiros, modal mostra estado de dados insuficientes.
 5. Se houver dados, `useFinancialAnalysis` dispara mutation.
 6. `financialAnalysisService` faz `POST /summary/analysis`.
 7. Enquanto aguarda, `FinancialAnalysisLoading` aparece.
 8. Em sucesso, `FinancialAnalysisResult` renderiza resumo, positivos, alertas e recomendações.
 9. Em erro, `ErrorState` permite tentar novamente.
-

32. Testes

Estratégia real:

Unitário

→ schemas e funções/componentes pequenos

Componente

→ páginas e componentes renderizados com Testing Library

Integração

→ requests interceptados por MSW

E2E

→ navegador real com Playwright

Testes relevantes encontrados:

Área	Arquivo	Cenários
Pessoas schema	src/features/people/schemas/personSchema.test.ts	pessoa válida, idade inválida
Pessoas página	src/features/people/pages/PeopleConsultPage/PeopleConsultPage.test.tsx	renderiza API, vazio, erro/retry, exclusão, feedback
Cadastro pessoa	src/features/people/pages/PersonRegisterPage/PersonRegisterPage.test.tsx	envia API, erro amigável
Transação schema	src/features/transactions/schemas/transactionSchema.test.ts	transação válida, valor negativo
Cadastro transação	src/features/transactions/pages/TransactionRegisterPage/TransactionRegisterPage.test.tsx	vazio sem pessoas, regra de menor
Lista transações	src/features/transactions/pages/TransactionsConsultPage/TransactionsConsultPage.test.tsx	loading, dados, vazio, erro/retry
Resumo service	src/features/summary/services/summaryService.test.ts	filtros por pessoa e período
Overview	src/features/summary/components/OverviewPanel.test.tsx	total geral
Resumo/IA	src/features/summary/pages/SummaryPage.test.tsx	filtros, loading IA, resultado, retry, dados insuficientes
Auth schemas	src/features/authentication/schemas/authSchemas.test.ts	login válido, confirmação divergente

Área	Arquivo	Cenários
Shared Table	src/shared/components/Table/Table.test.tsx	vazio, ação de linha
ErrorState	src/shared/components/ErrorState/ErrorState.test.tsx	conteúdo, retry, sem botão
PageHeader	src/shared/components/PageHeader/PageHeader.test.tsx	breadcrumb/título
PageTitle	src/shared/components/PageTitle/PageTitle.test.tsx	título do documento
E2E auth/responsivo/IA	tests/e2e/auth.spec.ts	redirect, bypass, overflow, Drawer mobile, IA

33. MSW

Arquivos:

- `src/test/server.ts`
- `src/test/setup.ts`
- specs com `server.use(...)`.

Fluxo nos testes:

```
Frontend
↓
request HTTP
↓
MSW intercepta
↓
resposta simulada
↓
componente atualiza UI
```

`src/test/setup.ts` liga o servidor antes de todos os testes, reseta handlers após cada teste e fecha no final. `onUnhandledRequest: "error"` ajuda a detectar chamadas não mockadas.

MSW é apenas para testes. Em runtime real, o frontend usa `VITE_API_URL` e espera uma API compatível.

34. Playwright

Arquivo: `playwright.config.ts`.

Configuração real:

- `testDir: './tests/e2e';`
- dev server: `npm run dev -- --host 127.0.0.1;`
- URL base: `http://127.0.0.1:5173;`
- `reuseExistingServer: true;`
- trace em primeira repetição;
- projetos desktop, tablet e mobile.

O arquivo `tests/e2e/auth.spec.ts` usa `VITE_BYPASS_AUTH` para decidir quais testes executar:

- bypass desligado: rota privada deve redirecionar para `/entrar`;
 - bypass ligado: rotas privadas podem ser navegadas sem sessão real;
 - mobile: abre/fecha navegação pelo Drawer;
 - desktop: fluxo de IA com `page.route` simulando `/api/people`, `/api/summary` e `/api/summary/analysis`.
-

35. CI/CD

Workflow real: `.github/workflows/ci.yml`.

Gatilhos:

- push na branch `main`;
- pull request.

Etapas:

```
Checkout
↓
Setup Node 24 com cache npm
↓
npm ci
↓
npm run lint
↓
npm test
↓
npm run build
```

O CI não roda Playwright E2E no workflow atual.

36. Variáveis De Ambiente

Arquivos:

- `.env.example`
- `src/shared/config/env.ts`
- `playwright.config.ts`

Variável	Objetivo	Exemplo
VITE_API_URL	Base URL da API REST	<code>http://localhost:8080/api</code>
VITE_BYPASS_AUTH	Libera rotas sem autenticação real em desenvolvimento/teste	<code>false</code>

Não há segredos no `.env.example`. Como variáveis `VITE_` são expostas ao frontend, não se deve colocar chaves secretas nelas.

37. Backend Futuro

O que depende do backend:

- API .NET/C# compatível com os endpoints configurados.
- PostgreSQL para persistência real.
- Geração definitiva de IDs.
- Autenticação real, sessão por cookie, segurança de senha.
- Autorização e proteção de rotas no servidor.
- Exclusão em cascata de transações ao excluir pessoa.
- Validação server-side da regra de menor de 18 anos.
- Filtros reais de transações e resumo.
- Cálculo do resumo por pessoa.
- Processamento de IA para análise financeira.

Separação de responsabilidades:

Frontend

→ experiência, formulários, feedback, chamada da API, cache

Backend

→ persistência, segurança, regra definitiva, consistência dos dados, IA

38. Endpoints Esperados

Todos os endpoints abaixo vêm de `src/shared/api/apiEndpoints.ts` e dos services.

Método	Endpoint	Finalidade	Service	Status
POST	<code>/auth/signin</code>	Entrar	<code>authenticationService.signIn</code>	Contrato provisório
POST	<code>/auth/signup</code>	Criar conta	<code>authenticationService.signUp</code>	Contrato provisório
POST	<code>/auth/signout</code>	Sair	<code>authenticationService.signOut</code>	Contrato provisório
GET	<code>/auth/me</code>	Usuário atual	<code>authenticationService.getCurrentUser</code>	Contrato provisório
GET	<code>/people</code>	Listar pessoas	<code>peopleService.getPeople</code>	Contrato provisório
GET	<code>/people/:id</code>	Buscar pessoa por id	<code>peopleService.getPersonById</code>	Contrato provisório
POST	<code>/people</code>	Criar pessoa	<code>peopleService.createPerson</code>	Contrato provisório
PUT	<code>/people/:id</code>	Atualizar pessoa	<code>peopleService.updatePerson</code>	Contrato provisório
DELETE	<code>/people/:id</code>	Excluir pessoa	<code>peopleService.deletePerson</code>	Contrato provisório
GET	<code>/transactions</code>	Listar transações com filtros opcionais	<code>transactionsService.getTransactions</code>	Contrato provisório
GET	<code>/transactions/:id</code>	Buscar transação por id	<code>transactionsService.getTransactionById</code>	Contrato provisório
POST	<code>/transactions</code>	Criar transação	<code>transactionsService.createTransaction</code>	Contrato provisório
PUT	<code>/transactions/:id</code>	Atualizar transação	<code>transactionsService.updateTransaction</code>	Contrato provisório
DELETE	<code>/transactions/:id</code>	Excluir transação	<code>transactionsService.deleteTransaction</code>	Contrato provisório
GET	<code>/summary</code>	Buscar resumo financeiro	<code>summaryService.getSummary</code>	Contrato provisório
POST	<code>/summary/analysis</code>	Solicitar análise financeira com IA	<code>financialAnalysisService.analyze</code>	Contrato provisório

39. Regras De Negócio

Regra	Frontend	Backend futuro
Pessoa possui nome, idade e e-mail	<code>personSchema</code> , <code>PersonRegisterPage</code>	persistir e validar unicidade quando aplicável
ID é automático	formulário não envia <code>id</code>	gerar no banco/API
Transação pertence a pessoa válida	<code>personId</code> obrigatório e seleção via <code>usePeople</code>	validar existência
Valor da transação > 0	<code>transactionSchema</code>	validar definitivamente
Tipo é Receita ou Despesa	<code>z.enum(["income", "expense"])</code>	validar contrato
Menor de 18 só Despesa	UI desabilita/corrigir e submit bloqueia	regra definitiva server-side
Saldo = receita - despesa	<code>PersonSummaryCard</code> , <code>OverviewPanel</code>	pode calcular/validar conforme contrato
Excluir pessoa remove transações	confirmação e invalidação de caches	exclusão em cascata
Total geral soma pessoas	<code>OverviewPanel</code>	endpoint separado não é obrigatório

40. Glossário

Termo	Explicação no contexto do projeto
Component	Bloco visual React, como <code>PageHeader</code> ou <code>PersonSummaryCard</code>
Hook	Função React para lógica reutilizável, como <code>usePeople</code>
Provider	Componente que disponibiliza contexto global, como <code>AuthProvider</code>
Context	Mecanismo React usado para sessão/authenticação
Query	Leitura assíncrona com cache, como listar pessoas
Mutation	Ação de escrita, como criar pessoa ou solicitar IA
Cache	Dados guardados pelo TanStack Query
DTO	Formato de request/response da API
Service	Camada que chama endpoints, como <code>peopleService</code>
Schema	Validação Zod de formulário
Route	URL declarada em <code>src/app/routes.tsx</code>
Guard	Componente que protege/libera rota, como <code>PrivateRoute</code>
Skeleton	Placeholder visual durante carregamento
MSW	Mock Service Worker usado para interceptar HTTP em testes
E2E	Teste de ponta a ponta em navegador real com Playwright

41. Guia: Como Estudar Este Projeto

1. Abra `src/main.tsx`: observe como a aplicação entra no React.
2. Abra `src/app/providers/AppProviders.tsx`: pergunte quais dependências globais toda página recebe.
3. Abra `src/app/routes.tsx`: acompanhe rotas públicas, privadas, lazy loading e 404.
4. Abra `src/app/routes/paths.ts`: veja a fonte única das URLs.
5. Abra `src/shared/components/layout/Template/Template.tsx`: entenda navegação, Drawer e `<Outlet />`.
6. Abra `src/shared/components/layout/AuthTemplate/AuthTemplate.tsx`: entenda login/cadastro.
7. Estude Pessoas começando por `src/features/people/pages/PeopleConsultPage/PeopleConsultPage.tsx`: veja loading, erro, vazio e tabela.
8. Abra `src/features/people/hooks/usePeople.ts`: diferencie query e mutation.
9. Abra `src/features/people/services/peopleService.ts`: veja a chamada real de API.
10. Abra `src/features/people/schemas/personSchema.ts`: veja validação de formulário.
11. Estude Transações por `src/features/transactions/pages/TransactionRegisterPage/TransactionRegisterPage.tsx`: foque em carregamento de pessoas e regra de idade.
12. Abra `src/features/transactions/schemas/transactionSchema.ts`: veja valor positivo e campos obrigatórios.
13. Estude Resumo por `src/features/summary/pages/SummaryPage.tsx`: veja filtros, query e IA.
14. Abra `src/features/summary/components/OverviewPanel.tsx`: veja cálculo do total geral.
15. Abra `src/features/summary/services/financialAnalysisService.ts`: veja o endpoint de IA e timeout.
16. Abra `src/features/authentication/providers/AuthProvider.tsx`: entenda sessão e bypass.
17. Abra `src/shared/api/httpClient.ts`: entenda base URL, timeout, credentials e erros.
18. Abra `src/shared/api/apiError.ts`: veja mensagens amigáveis.
19. Leia testes de Pessoas e Resumo para ver o comportamento esperado.
20. Leia `tests/e2e/auth.spec.ts` para entender bypass, responsividade e IA no navegador.

42. Guia De Depuração

Problema	Onde olhar primeiro	Arquivos
Página não abre	Rotas e guards	<code>src/app/routes.tsx</code> , <code>src/features/authentication/components/PrivateRoute.tsx</code> , <code>src/features/authentication/components/PublicRoute.tsx</code>
Redireciona errado	Auth e bypass	<code>src/features/authentication/providers/AuthProvider.tsx</code> , <code>src/shared/config/env.ts</code>
API não retorna	Service, endpoint e HTTP client	<code>src/shared/api/apiEndpoints.ts</code> , <code>src/shared/api/httpClient.ts</code> , service da feature
Form não envia	Schema, erros e submit	página do formulário, schema Zod
Dados não atualizam	Query key e invalidação	hooks da feature
Pessoa excluída mas resumo antigo	Invalidação de <code>["summary"]</code>	<code>src/features/people/hooks/usePeople.ts</code>
Transação criada mas lista antiga	Invalidação de <code>["transactions"]</code>	<code>src/features/transactions/hooks/useTransactions.ts</code>
Regra de menor falha	Watch, Select e submit	<code>src/features/transactions/pages/TransactionRegisterPage/TransactionRegisterPage.tsx</code>
Mensagem técnica aparece	Tratamento de erro	<code>src/shared/api/apiError.ts</code> , <code>src/shared/components/ErrorMessage/ErrorMessage.tsx</code>
Layout quebra mobile	SCSS e Drawer	<code>src/shared/components/layout/Template/Template.tsx</code> , <code>src/shared/components/layout/Template/Template.scss</code> , <code>src/styles/_breakpoints.scss</code>
IA não retorna	Mutation e endpoint	<code>src/features/summary/hooks/useFinancialAnalysis.ts</code> , <code>src/features/summary/services/financialAnalysisService.ts</code>
Teste HTTP falha	Handler MSW ausente	spec correspondente, <code>src/test/server.ts</code> , <code>src/test/setup.ts</code>

43. Como Adicionar Uma Nova Feature

Seguindo o padrão atual, uma feature como `categories` poderia nascer assim:

```
src/features/categories/  
├── components/  
├── hooks/  
├── pages/  
├── schemas/  
├── services/  
└── types/
```

Fluxo recomendado:

```
tipo de domínio  
↓  
DTO e mapper  
↓  
service  
↓  
hook TanStack Query  
↓  
schema Zod  
↓  
página/componente  
↓  
rota  
↓  
testes
```

Também seria necessário adicionar caminhos em `src/app/routes/paths.ts`, títulos em `src/app/routes/pageTitles.ts`, rotas em `src/app/routes.tsx` e item de navegação em `src/shared/components/layout/Template/Template.tsx`, caso a feature seja página principal.

44. Boas Práticas Encontradas

- Arquitetura por feature: facilita estudar e evoluir um domínio por vez.
 - TypeScript: contratos claros para entidades, DTOs, hooks e props.
 - Services isolados: componentes não conhecem detalhes de `fetch`.
 - HTTP client centralizado: base URL, timeout, credentials e erros em um lugar.
 - TanStack Query: cache e invalidação explícita após mutations.
 - Zod + React Hook Form: validação consistente com mensagens por campo.
 - Componentes compartilhados: `Table`, estados e skeletons reduzem repetição.
 - Lazy loading de páginas: rotas carregam sob demanda.
 - Estados de UI bem separados: loading, erro, vazio e sucesso.
 - Testes com MSW: valida comportamento sem depender de backend real.
 - Playwright: cobre navegação, responsividade e fluxo de IA.
 - CI: valida lint, testes e build em PR/push.
-

45. Pontos De Atenção

Esses pontos são naturais para a evolução do projeto e não significam defeito do frontend atual:

- Alinhar contratos com o backend .NET/C# antes da integração final.
 - Definir persistência PostgreSQL e geração de IDs.
 - Implementar autenticação real com cookies seguros.
 - Avaliar refresh token, expiração e proteção CSRF conforme estratégia do backend.
 - Revalidar no servidor a regra de menor de 18 anos.
 - Garantir exclusão em cascata no banco/API.
 - Definir política de erro e payloads de validação.
 - Implementar observabilidade/logs para análise financeira com IA.
 - Definir limites de uso, custo e segurança da IA.
 - Decidir se updates preparados nos services ganharão telas próprias.
 - Decidir se botões sociais de Google/GitHub serão integrados ou removidos até existirem.
-

46. Checklist De Compreensão

- ☐ Sei onde a aplicação inicia.
- ☐ Sei qual provider entrega TanStack Query.
- ☐ Sei onde o tema MUI é configurado.
- ☐ Sei onde as rotas são declaradas.
- ☐ Sei diferenciar rota pública e privada.
- ☐ Sei como `VITE_BYPASS_AUTH` altera o comportamento.
- ☐ Sei como uma pessoa é cadastrada.
- ☐ Sei por que o formulário de pessoa não envia `id`.
- ☐ Sei como pessoas são listadas.
- ☐ Sei como uma pessoa é excluída.
- ☐ Sei onde a exclusão em cascata deve ser garantida.
- ☐ Sei como uma transação é cadastrada.
- ☐ Sei como funciona a regra de menor de 18 anos.
- ☐ Sei onde `income` vira "Receita" e `expense` vira "Despesa".
- ☐ Sei como o resumo usa filtros.
- ☐ Sei por que filtros entram na query key.
- ☐ Sei como o total geral é calculado.
- ☐ Sei como uma query funciona.
- ☐ Sei como uma mutation invalida caches.
- ☐ Sei como o frontend chama a API.
- ☐ Sei onde ficam DTOs e mappers.
- ☐ Sei como os testes usam MSW.
- ☐ Sei o que Playwright cobre.
- ☐ Sei o que ainda depende do backend.
- ☐ Sei como a análise financeira com IA está preparada.

47. Apêndice: Mapa De Arquivos Importantes

Arquivo	Responsabilidade	Leia quando quiser entender
src/main.tsx	Entrada React	bootstrap da aplicação
src/app/providers/AppProviders.tsx	Providers globais	contexto global
src/app/routes.tsx	Rotas e lazy loading	navegação
src/app/routes/paths.ts	Constantes de URL	caminhos
src/app/routes/pageTitles.ts	Títulos de página	document.title
src/app/query/queryClient.ts	Config do TanStack Query	cache/retry
src/app/theme/theme.ts	Tema Material UI	tokens visuais
src/shared/config/env.ts	Variáveis de ambiente	API/bypass
src/shared/api/apiEndpoints.ts	Endpoints REST	contrato esperado
src/shared/api/httpClient.ts	Cliente HTTP	fetch/timeout/credentials
src/shared/api/apiError.ts	Erros amigáveis	tratamento de falhas
src/shared/components/layout/Template/Template.tsx	Layout privado	navegação principal
src/shared/components/layout/AuthTemplate/AuthTemplate.tsx	Layout público	login/cadastro
src/shared/components/Table/Table.tsx	Tabela genérica	listagens
src/shared/components/FeedbackState/FeedbackState.tsx	Estado visual base	erro/404
src/shared/components/ErrorResponse/ErrorResponse.tsx	Erro com ação	falhas
src/shared/components/DataState/EmptyState.tsx	Estado vazio	listas/resumo
src/features/authentication/providers/AuthProvider.tsx	Sessão	auth
src/features/authentication/services/authenticationService.ts	API de auth	login/logout
src/features/authentication/schemas/authSchemas.ts	Validação auth	forms auth
src/features/people/pages/PeopleConsultPage/PeopleConsultPage.tsx	Listagem pessoas	consulta/exclusão
src/features/people/pages/PersonRegisterPage/PersonRegisterPage.tsx	Cadastro pessoa	form pessoa

Arquivo	Responsabilidade	Leia quando quiser entender
src/features/people/hooks/usePeople.ts	Queries/mutations pessoas	cache pessoas
src/features/people/services/peopleService.ts	API pessoas	endpoints pessoas
src/features/people/schemas/personSchema.ts	Validação pessoa	regra de formulário
src/features/transactions/pages/TransactionRegisterPage/TransactionRegisterPage.tsx	Cadastro transação	regra de menor
src/features/transactions/pages/TransactionsConsultPage/TransactionsConsultPage.tsx	Listagem transações	consulta/exclusão
src/features/transactions/hooks/useTransactions.ts	Queries/mutations transações	cache transações
src/features/transactions/services/transactionsService.ts	API transações	endpoints transações
src/features/transactions/schemas/transactionSchema.ts	Validação transação	campos obrigatórios
src/features/summary/pages/SummaryPage.tsx	Resumo e IA	fluxo financeiro
src/features/summary/components/SummaryFilters.tsx	Filtros	pessoa/período
src/features/summary/components/PersonSummaryCard.tsx	Card individual	saldo por pessoa
src/features/summary/components/OverviewPanel.tsx	Total geral	somatórios
src/features/summary/services/summaryService.ts	API resumo	filtros HTTP
src/features/summary/services/financialAnalysisService.ts	API IA	análise financeira
src/test/setup.ts	Setup MSW	testes
tests/e2e/auth.spec.ts	E2E	auth/responsividade/IA
.github/workflows/ci.yml	CI	pipeline

48. Apêndice: Fluxos Resumidos

CADASTRO DE PESSOA

```
PersonRegisterPage
→ personSchema
→ useCreatePerson
→ peopleService.createPerson
→ httpClient.post
→ POST /people
→ invalidate ["people"]
→ PeopleConsultPage
```

TRANSAÇÃO

```
TransactionRegisterPage
→ usePeople
→ regra de idade
→ transactionSchema
→ useCreateTransaction
→ transactionsService.createTransaction
→ POST /transactions
→ invalidate ["transactions"] e ["summary"]
```

RESUMO

```
SummaryFilters
→ filters state
→ useSummary(["summary", filters])
→ summaryService.getSummary
→ GET /summary
→ PersonSummaryCard
→ OverviewPanel
```

AUTH

```
SignIn/SignUp
→ authSchemas
→ AuthProvider.signIn/signUp
→ authenticationService
→ /auth/*
→ user em contexto
→ PrivateRoute/PublicRoute
```

IA

AiAnalysisCard

- FinancialAnalysisDialog
- useFinancialAnalysis
- financialAnalysisService.analyze
- POST /summary/analysis
- FinancialAnalysisLoading
- FinancialAnalysisResult